

Shader Math Pocket Guide

1. Vector Math

Dot Product:

$$\text{dot}(\mathbf{a}, \mathbf{b}) = a_x b_x + a_y b_y + a_z b_z$$

GLSL: `float d = dot(a, b);`

Cross Product (3D only):

$$\text{cross}(\mathbf{a}, \mathbf{b}) = (a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x)$$

GLSL: `vec3 c = cross(a, b);`

Normalize:

$$\text{normalize}(\mathbf{v}) = \mathbf{v} / \text{length}(\mathbf{v})$$

GLSL: `vec3 n = normalize(v);`

Length:

$$\text{length}(\mathbf{v}) = \sqrt{x^2 + y^2 + z^2}$$

GLSL: `float l = length(v);`

2. Transformations

Translation:

$$\mathbf{p}' = \mathbf{p} + \mathbf{t}$$

Scaling:

$$\mathbf{p}' = \mathbf{s} * \mathbf{p}$$

Rotation (2D):

$$\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

3D Rotation: Use rotation matrices or quaternions.

GLSL Example:

`vec4 pos = Projection * View * Model * vec4(position, 1.0);`

3. Lighting Models

Lambert Diffuse:

$$I = k_d * \max(0, \text{dot}(\mathbf{n}, \mathbf{l}))$$

Phong Specular:

$$I = k_s * \text{pow}(\max(0, \text{dot}(\mathbf{r}, \mathbf{v})), \alpha)$$

Reflection Vector:

$$\mathbf{r} = \mathbf{i} - 2 * \text{dot}(\mathbf{n}, \mathbf{i}) * \mathbf{n}$$

GLSL Example:

`float diff = max(dot(normal, lightDir), 0.0);`

`vec3 reflectDir = reflect(-lightDir, normal);`

4. Interpolation & Curves

Linear Interpolation:

$\text{mix}(a, b, t) = a \cdot (1-t) + b \cdot t$

Smoothstep:

$\text{smoothstep}(e0, e1, x) =$
 $t = \text{clamp}((x-e0)/(e1-e0), 0, 1)$
 $\text{return } t \cdot t \cdot (3 - 2 \cdot t);$

5. Oscillation & Animation

Sine Wave:

$y = A \cdot \sin(\omega \cdot t + \phi)$

GLSL Example:

$\text{float wave} = \sin(\text{time} \cdot \text{frequency} + \text{phase});$

6. Procedural Geometry (SDF)

Sphere SDF:

$d = \text{length}(p - c) - r$

Plane SDF:

$d = \text{dot}(p - p0, n)$

7. Random & Noise

GLSL Hash Random:

```
float rand(vec2 co) {  
    return fract(sin(dot(co.xy, vec2(12.9898, 78.233))) * 43758.5453);  
}
```

Perlin/Simplex Noise:

Used for smooth procedural patterns.

Tips:

- Keep vectors normalized for lighting math.
- Avoid heavy branching in shaders.
- Use built-in GPU functions where possible.